

```
"""
```

```
 FOOTBALL TRADING ALERT BOT - V3  
SofaScore + The Odds API → Alertes Telegram
```

```
INSTALLATION (une seule fois) :  
pip install requests schedule
```

```
LANCEMENT :  
python football_alert_bot_v3.py  
"""
```

```
import requests  
import schedule  
import time  
from datetime import datetime
```

```
# =====  
#  CONFIGURATION  
# =====
```

```
TELEGRAM_TOKEN = "8601521492:AAGx10bdhu3UeEMAfKh0NlrdDUcxbLK85o8"  
CHAT_ID        = "351609302"  
ODDS_API_KEY   = "e610e4b32d5038e6c54a1316a571cda0" # The Odds API
```

```
# =====  
#  CRITÈRES D'ALERTE  
# =====
```

```
CRITERES = {  
    "minute_min": 75,      # Alerte après la 75e minute  
    "tirs_favori_min": 3,   # Minimum 3 tirs du favori  
    "indice_pression_min": 65, # Indice de pression minimum /100  
    "cote_favori_live_min": 2.0, # Cote du favori >= 2.0 (value)  
    "ligues_surveillees": [  
        "Premier League",  
        "La Liga",  
        "Serie A",  
        "Bundesliga",  
        "Ligue 1",  
        "UEFA Champions League",  
        "UEFA Europa League",  
    ],  
}
```

```

POIDS_PRESSION = {
    "tirs_cadres":          30,
    "corners":              20,
    "possession":           20,
    "attaques_dangereuses": 30,
}

# Clés de ligues pour The Odds API
ODDS_LEAGUES = [
    "soccer_epl",
    "soccer_spain_la_liga",
    "soccer_italy_serie_a",
    "soccer_germany_bundesliga",
    "soccer_france_ligue_one",
    "soccer_uefa_champs_league",
    "soccer_uefa_europa_league",
]

INTERVALLE_SCAN = 30 # secondes

# =====
# 📡 HEADERS SOFASCORE
# =====

HEADERS_SOFA = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36",
    "Accept":     "application/json",
    "Referer":    "https://www.sofascore.com/",
}

alertes_envoyees = set()
cache_odds       = {} # cache des cotes pour éviter trop de requêtes API

# =====
# 🏆 THE ODDS API – COTES EN DIRECT
# =====

def get_odds_pour_match(home_team: str, away_team: str) -> dict:
    """
    Récupère les cotes live depuis The Odds API pour un match donné.
    Retourne : { home_cote, away_cote, draw_cote, favori_home }
    """
    # Vérifier le cache (valable 60 secondes)
    cle_cache = f"{home_team}_{away_team}"
    if cle_cache in cache_odds:
        ts, data = cache_odds[cle_cache]

```

```

if time.time() - ts < 60:
    return data

for league_key in ODDS_LEAGUES:
    url = (
        f"https://api.the-odds-api.com/v4/sports/{league_key}"
        f"/odds/?apiKey={ODDS_API_KEY}&regions=eu&markets=h2h&oddsFormat=decim"
    )
    try:
        r = requests.get(url, timeout=10)
        if r.status_code != 200:
            continue

        for event in r.json():
            h = event.get("home_team", "").lower()
            a = event.get("away_team", "").lower()

            # Correspondance partielle sur le nom d'équipe
            if (home_team.lower()[:5] in h or h[:5] in home_team.lower()) and
                (away_team.lower()[:5] in a or a[:5] in away_team.lower()):

                bookmakers = event.get("bookmakers", [])
                if not bookmakers:
                    continue

                # Prendre le premier bookmaker dispo
                for bk in bookmakers:
                    for market in bk.get("markets", []):
                        if market.get("key") != "h2h":
                            continue

                    outcomes = {o["name"]: o["price"] for o in market.get
                        "outcomes", []}
                    home_cote = outcomes.get(event["home_team"], 0.0)
                    away_cote = outcomes.get(event["away_team"], 0.0)
                    draw_cote = outcomes.get("Draw", 0.0)

                    favori_home = (home_cote <= away_cote) if home_cote >
                        draw_cote else False

                    result = {
                        "home_cote": round(home_cote, 2),
                        "away_cote": round(away_cote, 2),
                        "draw_cote": round(draw_cote, 2),
                        "favori_home": favori_home,
                        "bookmaker": bk.get("title", ""),
                    }

                    cache_odds[cle_cache] = (time.time(), result)
    except:
        pass
    return result

```

```

except Exception as e:
    print(f"⚠ Odds API erreur ({league_key}) : {e}")
    continue

return {}

# =====
# 📡 SOFAScore - MATCHS EN DIRECT
# =====

def get_matches_live() -> list:
    url = "https://api.sofascore.com/api/v1/sport/football/events/live"
    try:
        r = requests.get(url, headers=HEADERS_SOFA, timeout=15)
        return r.json().get("events", []) if r.status_code == 200 else []
    except Exception as e:
        print(f"❌ SofaScore : {e}")
        return []

def get_stats_match(event_id: int) -> dict:
    """Stats complètes d'un match depuis SofaScore."""
    url = f"https://api.sofascore.com/api/v1/event/{event_id}/statistics"
    stats = {
        "tirs_home": 0,      "tirs_away": 0,
        "tirs_cadres_home": 0, "tirs_cadres_away": 0,
        "corners_home": 0,    "corners_away": 0,
        "possession_home": 50, "possession_away": 50,
        "attaques_dang_home": 0, "attaques_dang_away": 0,
    }
    try:
        r = requests.get(url, headers=HEADERS_SOFA, timeout=10)
        if r.status_code != 200:
            return stats

        for groupe in r.json().get("statistics", []):
            for grp in groupe.get("groups", []):
                for item in grp.get("statisticsItems", []):
                    nom = item.get("name", "").lower()
                    h = _safe_int(item.get("home"))
                    a = _safe_int(item.get("away"))

                    if "total shots" in nom or "shots total" in nom:
                        stats["tirs_home"] = h;          stats["tirs_away"] = a
                    elif "shots on target" in nom:

```

```

        stats["tirs_cadres_home"] = h; stats["tirs_cadres_away"]
    elif "corner" in nom:
        stats["corners_home"] = h; stats["corners_away"] = a
    elif "possession" in nom:
        stats["possession_home"] = h; stats["possession_away"] =
    elif "dangerous" in nom or "attack" in nom:
        stats["attaques_dang_home"] = h; stats["attaques_dang_awa
except Exception as e:
    print(f"⚠ Stats {event_id} : {e}")
return stats

def _safe_int(val) -> int:
    try:
        return int(str(val).replace("%", "").strip() or 0)
    except:
        return 0

# =====
# 🏠 INDICE DE PRESSION (0-100)
# =====

def calcul_indice_pression(stats: dict, favori_home: bool) -> float:
    side = "home" if favori_home else "away"
    opp = "away" if favori_home else "home"

    def ratio(a, b): return (a / (a + b)) * 100 if (a + b) > 0 else 50

    score_tc = ratio(stats[f"tirs_cadres_{side}"], stats[f"tirs_cadres_{opp}"])
    score_corn = ratio(stats[f"corners_{side}"], stats[f"corners_{opp}"])
    score_poss = stats[f"possession_{side}"]
    score_ad = ratio(stats[f"attaques_dang_{side}"], stats[f"attaques_dang_{opp}

    indice = (
        score_tc * POIDS_PRESSION["tirs_cadres"] / 100 +
        score_corn * POIDS_PRESSION["corners"] / 100 +
        score_poss * POIDS_PRESSION["possession"] / 100 +
        score_ad * POIDS_PRESSION["attaques_dangereuses"] / 100
    )
    return round(indice, 1)

def emoji_pression(i: float) -> str:
    if i >= 80: return "🔥🔥🔥"
    if i >= 70: return "🔥🔥"
    return "🔥"

```

```

# =====
# 🧠 ANALYSE D'UN MATCH
# =====

def analyser_match(event: dict):
    try:
        event_id = event.get("id")
        status = event.get("status", {})
        type_status = status.get("type", "")
        minute_raw = status.get("description", "0")

        if type_status != "inprogress":
            return None

        # Minute
        try:
            minute_int = int(str(minute_raw).split("+")[0].strip())
        except:
            minute_int = 0

        if minute_int < CRITERES["minute_min"]:
            return None

        # Ligue
        ligue = event.get("tournament", {}).get("name", "?")
        if CRITERES["ligues_surveillees"]:
            if not any(l.lower() in ligue.lower() for l in CRITERES["ligues_surve

        # Équipes & score
        home_team = event.get("homeTeam", {}).get("name", "?")
        away_team = event.get("awayTeam", {}).get("name", "?")
        home_score = event.get("homeScore", {}).get("current", 0)
        away_score = event.get("awayScore", {}).get("current", 0)

        # — Cotes via The Odds API —————
        odds = get_odds_pour_match(home_team, away_team)

        if odds:
            favori_home = odds["favori_home"]
            cote_favori = odds["home_cote"] if favori_home else odds["away_cot
            cote_outsider = odds["away_cote"] if favori_home else odds["home_cot
            draw_cote = odds["draw_cote"]
            bookmaker = odds["bookmaker"]
        else:

```

```

# Fallback si pas de cotes dispo : score comme proxy
favori_home    = True
cote_favori    = 0.0
cote_outsider  = 0.0
draw_cote      = 0.0
bookmaker      = "N/A"

# Filtre cote minimum (value)
if cote_favori > 0 and cote_favori < CRITERES["cote_favori_live_min"]:
    return None

# — Scénario valide : nul ou favori mené —————
match_nul      = (home_score == away_score)
if favori_home:
    favori_mene    = (home_score < away_score)
    score_favori   = home_score
    score_outsider = away_score
    favori_name    = home_team
    outsider_name  = away_team
else:
    favori_mene    = (away_score < home_score)
    score_favori   = away_score
    score_outsider = home_score
    favori_name    = away_team
    outsider_name  = home_team

if not match_nul and not favori_mene:
    return None

# — Stats SofaScore —————
stats          = get_stats_match(event_id)
tirs_favori    = stats["tirs_home"] if favori_home else stats["tirs_away"]

if tirs_favori < CRITERES["tirs_favori_min"]:
    return None

indice = calcul_indice_pression(stats, favori_home)
if indice < CRITERES["indice_pression_min"]:
    return None

#  Tous les critères remplis
return {
    "id":            event_id,
    "home":          home_team,
    "away":          away_team,
    "favori":        favori_name,
    "outsider":      outsider_name,

```

```

        "score_str":          f"{home_score}-{away_score}",
        "score_favori":      score_favori,
        "score_outsider":    score_outsider,
        "minute":           minute_raw,
        "ligue":             ligue,
        "match_nul":         match_nul,
        "favori_mene":       favori_mene,
        "indice_pression":   indice,
        "tirs_favori":       tirs_favori,
        "corners_favori":    stats["corners_home"] if favori_home else stats["
        "possession_favori": stats["possession_home"] if favori_home else sta
        "attaques_dang":     stats["attaques_dang_home"] if favori_home else s
        "cote_favori":       cote_favori,
        "cote_outsider":     cote_outsider,
        "draw_cote":         draw_cote,
        "bookmaker":         bookmaker,
        "url":               f"https://www.sofascore.com/event/{event_id}",
    }

except Exception as e:
    print(f"⚠ Analyse : {e}")
    return None

# =====
# 📢 FORMATAGE ALERTE TELEGRAM
# =====

def formater_alerte(m: dict) -> str:
    heure = datetime.now().strftime("%H:%M:%S")

    scenario = (
        f"⚠ <b>{m['favori']} EST MENÉ</b> - cherche le retour"
        if m["favori_mene"] else
        f"👊 Match nul - <b>{m['favori']}</b> pousse pour gagner"
    )

    if m["cote_favori"] > 0:
        cotes_line = (
            f"📈 <b>Cotes ({m['bookmaker']})</b>\n"
            f"    {m['favori']} : <b>{m['cote_favori']}</b>  "
            f"Nul : <b>{m['draw_cote']}</b>  "
            f"{m['outsider']} : <b>{m['cote_outsider']}</b>\n"
        )
    else:
        cotes_line = "📉 Cotes : <i>vérifier manuellement</i>\n"

```



```

return (
    f"⚽ <b>ALERTE TRADING</b> - {heure}\n"
    f"_____ \n"
    f"🏆 <b>{m['ligue']}</b>\n"
    f"🆚 <b>{m['home']}</b> vs <b>{m['away']}</b>\n"
    f"🇫🇷 Score : <b>{m['score_str']}</b> | 🕒 <b>{m['minute']}</b>\n"
    f"_____ \n"
    f"{scenario}\n"
    f"_____ \n"
    f"{emoji_pression(m['indice_pression'])} <b>Pression : {m['indice_pression']}</b>\n"
    f"🎯 Tirs favori : <b>{m['tirs_favori']}</b>\n"
    f"📐 Corners : <b>{m['corners_favori']}</b>\n"
    f"⚡ Possession : <b>{m['possession_favori']}%</b>\n"
    f"💣 Attaques dang. : <b>{m['attaques_dang']}</b>\n"
    f"_____ \n"
    f"{cotes_line}"
    f"_____ \n"
    f"🔗 <a href='{m['url']}'>Voir sur SofaScore</a>\n"
    f"⚠️ <i>Max 2% de ta bankroll par trade</i>"
)

# =====
# 📱 ENVOI TELEGRAM
# =====

def envoyer_telegram(message: str):
    url = f"https://api.telegram.org/bot{TELEGRAM_TOKEN}/sendMessage"
    try:
        r = requests.post(url, json={
            "chat_id": CHAT_ID,
            "text": message,
            "parse_mode": "HTML",
            "disable_web_page_preview": True,
        }, timeout=10)
        print("✅ Alerte envoyée !" if r.status_code == 200 else f"❌ Telegram : ")
    except Exception as e:
        print(f"❌ Telegram : {e}")

# =====
# 🔄 BOUCLE PRINCIPALE
# =====

def scanner():
    print(f"\n🔍 Scan... {datetime.now().strftime('%H:%M:%S')}")
    matchs = get_matches_live()

```

```

print(f"    → {len(matches)} matchs live trouvés")

count = 0
for event in matches:
    match = analyser_match(event)
    if not match:
        continue

    cle = f"{match['id']}_{match['score_str']}_{match['minute']}"
    if cle in alertes_envoyees:
        continue

    envoyer_telegram(formater_alerte(match))
    alertes_envoyees.add(cle)
    count += 1
    time.sleep(1.5)

print(f"    → {count} alerte(s) envoyée(s)" if count else "    → Aucune opportu

def main():
    print("=" * 50)
    print("⚽ FOOTBALL TRADING ALERT BOT - V3")
    print("=" * 50)
    print(f"📡 The Odds API connectée")
    print(f"🕒 Minute déclencheur : {CRITERES['minute_min']}")
    print(f"🔥 Indice pression min : {CRITERES['indice_pression_min']/100")
    print(f"📈 Cote favori min : {CRITERES['cote_favori_live_min']}")
    print("=" * 50)

    if "TON_TOKEN" in TELEGRAM_TOKEN:
        print("\n⚠️ Configure TELEGRAM_TOKEN et CHAT_ID d'abord !")
        print("    1. Telegram → @BotFather → /newbot → copie le token")
        print("    2. Telegram → @userinfobot → copie ton ID")
        return

    envoyer_telegram(
        f"🤖 <b>Bot Football Trading V3 démarré !</b>\n"
        f"📡 Cotes via The Odds API\n"
        f"🕒 Scan toutes les {INTERVALLE_SCAN}s\n"
        f"🎯 Alertes dès la {CRITERES['minute_min']}e minute"
    )

    scanner()
    schedule.every(INTERVALLE_SCAN).seconds.do(scanner)

    print("\n✅ Bot actif ! Ctrl+C pour stopper.\n")

```

```
while True:
    schedule.run_pending()
    time.sleep(1)

if __name__ == "__main__":
    main()
```